漫步数码时代，

铸就精彩人生，

程序员修炼之路，

从精雕算法开始……

第2章 分治策略

(Divide and Conquer)

(三)

主讲人：周正深

1. 查找算法 *search*()

设A是具有n个元素的无序数组，若数组中存在元素x返回其在数组中的位置，若不存在返回-1。

分析：

蛮力法：遍历A，若存在x返回其位置，否则返回-1。$O(n)$——最优算法

伪代码：

```
for i←0 to n-1 do
    if A[i]==x then
        break;
if i==n then
    i←-1
return i
```

C++实现代码：

```
#include <iostream>
using namespace std;
int search(int A[], int x, int n){
    int i;
    for(i=0; i<n; i++){
        if( A[i]==x ) break;
    }
    if( i==n ) i=-1;
    return i;
}
int main(){
    int A[] = {3, 8, 17, 27, 28, 33, 98};
    int x = 33;
    int n = sizeof(A) / sizeof(int);
    int i = search(A, x, n);
    cout << "i = " << i << endl;
    return 0;
}
```

2. 折半查找算法 *halfSearch*()

设A是具有n个元素的有序数组，若数组中存在元素x返回其在数组中的位置，若不存在返回-1。

分析：

折半法：确定查找的起始位置l和结束位置r，若$l>r$，返回-1；否则取l和r的中间位置q中元素和x比较，若$A[q]==x$，返回位置q；若$A[q]>x$，$r=q-1$；否则，$l=q+1$。 $O(n)$--最优算法

伪代码：

```
l←0, r←n-1
while l<=r do
    q ← (l+r)/2
    if A[q]==x then return q;
    else if A[q]>x then r←q-1
    else l←q+1
return -1
```

C++实现代码：

```
#include <iostream>
using namespace std;
int halfSearch(int A[], int x, int n){
    int l=0, r=n-1, q;
    while( l<=r ){
        q = (l+r)/2;
        if( A[q]==x ) return q;
        else if( A[q]>x ) r=q-1;
        else l=q+1;
    }
    return -1;
}
int main(){
    int A[] = {3, 8, 17, 27, 28, 33, 98};
    int x = 33;
    int n = sizeof(A) / sizeof(int);
    int i = search(A, x, n);
    cout << "i = " << i << endl;
    return 0;
}
```

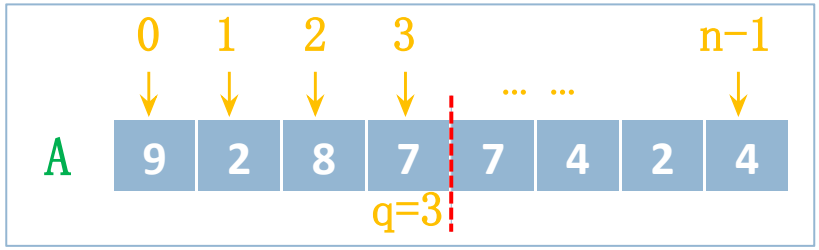
3. 二分归并排序 *binaryMergeSort*()

对有n个元素的数组A进行排序二分归并排序。

分析：

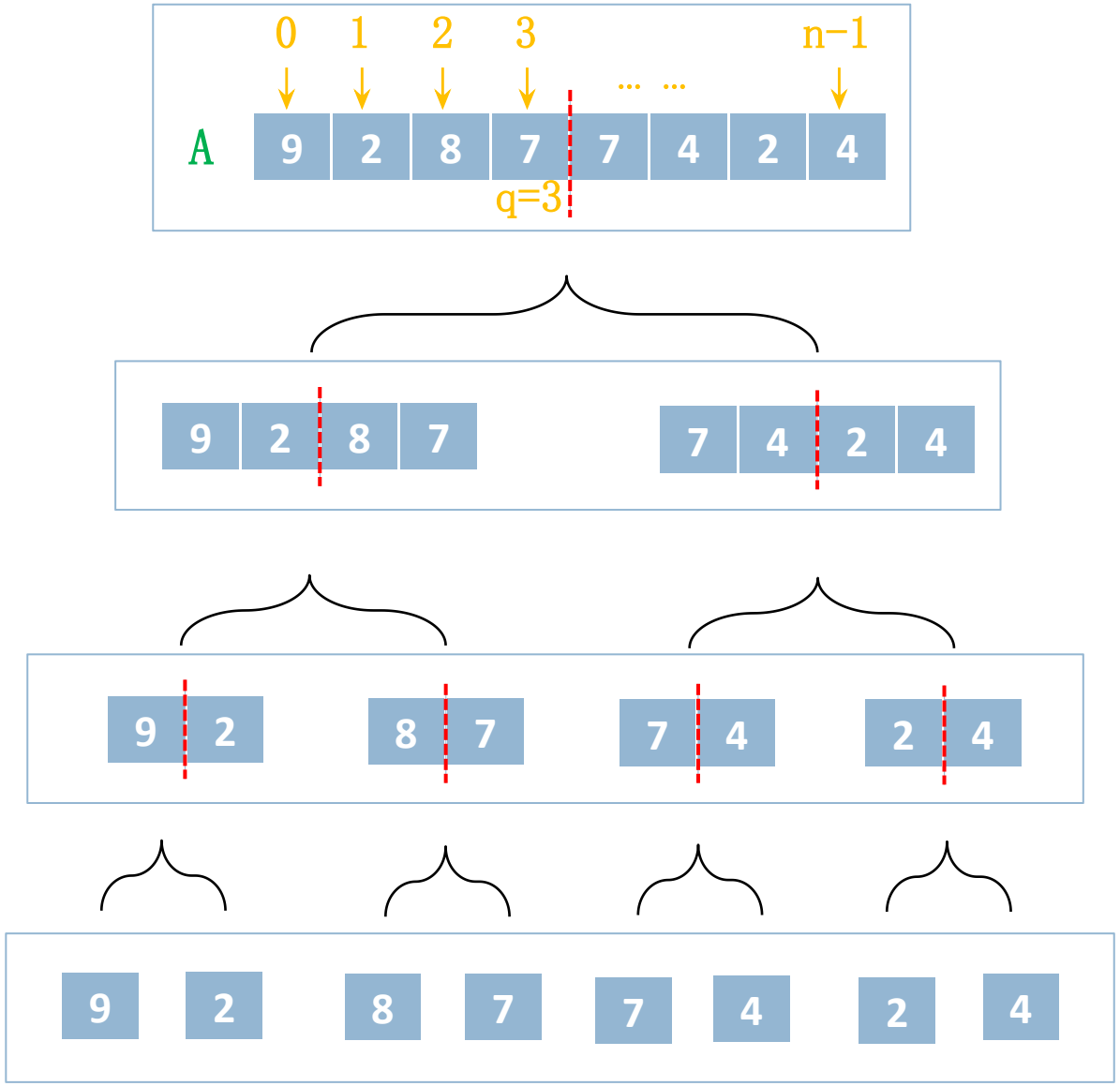
二分归并排序：如果n=1，算法结束。否则将数组A从中间分成左右两部分，分别对左部分和右部分进行二分归并排序，再将两部分有序数组合并成整体有序。 $O(n\log n)$ （空间复杂度$O(n)$）

l←0，r←n-1，q←(l+r)/2



习题课

$l \leftarrow 0$, $r \leftarrow n-1$, $q \leftarrow (l+r)/2$



伪代码：

```
void binaryMergeSort(int A[], int left, int right)
    if left>=right then return
    q ← (left+right)/2
    binaryMergeSort(A, left, q)
    binaryMergeSort(A, q+1, right)
    mergeSort(A, left, q, right)

void mergeSort(int A[], int left, int q, int right)
    复制A数组left~q到B数组，复制A数组q+1~right到C数组
    i←0, j←0, k←left;
    while i<q-left+1 && j<right-q do
        if B[i]<=C[j] then A[k]←B[i], k←k+1, i←i+1
        else A[k++]←C[j++], k←k+1, j←j+1
    }
    if i==q-left+1 then while j<right-q do A[k]=C[j++],k←k+1,j←j+1
    else while i<q-left+1 do A[k]=B[i], k←k+1, i←i+1
}
```

C++实现代码：

```
void mergeSort(int A[], int left, int q, int right){
    int B[q-left+1], C[right-q];
    for(int i=0; i<=q-left; i++)B[i]=A[left+i];
    for(int i=0; i<=right-q-1; i++)C[i]=A[q+1+i];
    int i=0, j=0, k=left;
    while(i<q-left+1 && j<right-q){
        if(B[i]<=C[j])A[k++]=B[i++];
        else A[k++]=C[j++];
    }
    if(i==q-left+1)while(j<right-q)A[k++]=C[j++];
    else while(i<q-left+1)A[k++]=B[i++];
}
```

C++实现代码：

```
#include <iostream>
using namespace std;

void mergeSort(int A[], int left, int q, int right);

void binaryMergeSort(int A[], int left, int right){
    if(left>=right)return;
    int q = (left+right)/2;
    binaryMergeSort(A, left, q);
    binaryMergeSort(A, q+1, right);
    mergeSort(A, left, q, right);
}

int main(){
    int A[] = {17, 4, 98, 4, 33, 7, 8, 9, 3, 2, 8, 4, 27, 28, 7};
    int n = sizeof(A)/sizeof(int);
    binaryMergeSort(A, 0, n-1);
    for (int i=0; i<n; i++)cout << A[i] << ", ";
    cout << endl;
    return 0;
}
```

4. 研究生试题：

问题描述：给定一个数组，计算数组中的逆序对数量。逆序对是指数组中两个元素i和j，如果i<j但是a[i]>a[j]，则称它们为一个逆序对。

(1)给出算法的设计思想。

(2)给出算法复杂度。

(3)根据设计思想，采用c或c++或java语言描述算法。

分析：

（一）蛮力法：

遍历数组每一个元素，分别找出前边比它小的数的个数。复杂度$O(n^2)$

（二）分治法：二分归并排序可以在$O(n\log n)$的时间复杂度内解决排序问题，当然也可以解决此问题。具体算法如下：

算法reverseOrder(A, left, right):

1. 将数组分成两半，分别求出两个子数组中的逆序对数量。
2. 将两个子数组进行合并，合并的过程中统计逆序对的数量。
3. 返回子数组的逆序对数量之和即可。

复杂度分析：

算法用时：$T(n)=2T(n/2)+O(n)$

其中 $a=2$, $b=2$, $f(n)=O(n)$

∵ $n^{\log_b a}=n^{\log_2 2}=n$ 与$f(n)=O(n)$相比 $\varepsilon=0$

符合主定理第2种情况

∴ 所以算法复杂度为 $n^{\log_b a}\log n = n\log n$

算法复杂度：$O(n\log n)$

伪代码：

```
reverseOrder( A, left, right):
    if left >= right then return 0
    q ← (left + right) / 2
    countLeft ← reverseOrder(A, left, q)
    countRight ← reverseOrder(A, q+1, right)
    countMerge ← merge(A, left, q, right)
    return countLeft + countRight + countMerge

merge( A, left, q, right):
    m←q-left+1, n←right-q
    B←A[left,q], C←A[q+1,right]
    count←0, i←0, j←0, k←left
    while i<m && j<n:
        if B[i] <= C[j] then A[k]←B[i],i←i+1,k←k+1
        else A[k]←C[j],j←j+1,k←k+1, count←count+(q+1-i)
    if i==m then while j<n do A[k]←C[j],j←j+1,k←k+1
    else while i<m do A[k]←B[i],i←i+1,k←k+1
    return count
```

C++实现代码：

```
int reverseOrder(int A[], int left, int right){
    if( left >= right ) return 0;
    int q = (left + right) / 2;
    int countLeft = reverseOrder(A, left, q);
    int countRight = reverseOrder(A, q+1, right);
    int countMerge = merge(A, left, q, right);
    return countLeft + countRight + countMerge;
}
```

C++实现代码：

```
int merge(int A[], int left, int q, int right){
    int m=q-left+1, n=right-q;
    int B[m], C[n];
    for(int i=0;i<m;i++)B[i]=A[left+i];
    for(int i=0;i<n;i++)C[i]=A[q+1+i];
    int count=0, i=0, j=0, k=left;
    while( i<m && j<=n ){
        if( B[i] <= C[j] ){
            A[k] = B[i];
            i++; k++;
        }
        else{
            A[k] = C[j];
            j++; k++;
            count += q+1-i;
        }
    }
    if(i==m)while(j<n)A[k++] = C[j++];
    else while(i<m)A[k++] = B[i++];
    return count;
```

C++实现代码：

```
#include <iostream>
using namespace std;
int merge(int A[], int left, int q, int right);
int reverseOrder(int A[], int left, int right);
int main(){
    int A[] = {5, 7, 4, 6}; // 调用示例
    int n = sizeof(A)/sizeof(int);
    int count = reverseOrder( A, 0, n-1 );
    cout << "逆序对数量：" << count << endl;
    return 0;
}
```

5. P48 8 哈工大考研试题：

设A是具有n个元素的升序数组，满足A[n-1]>...>A[1]>A[0]，那么A的中位数被定义为A[k]，其中k=(n-1)/2。如升序数组B=[2, 4, 4, 4, 7, 7, 8, 9]的中位数B[3]=4，C=[3, 8, 17, 27, 28, 33, 98]的中位数C[3]=27。给出快速算法，对任意给定包含元素个数分别为n(n>0)和m(m>0)的升序数组S[n]和T[m]，找到并返回S和T归并后的升序数组的中位数。

(1)给出算法的设计思想。

(2)使用c、c++或java语言给出相关数据类型定义。

(3)根据设计思想，采用c或c++或java语言描述算法。

分析：

（一）蛮力法：遍历B、C每一个元素，求AB数组中比它小的元素个数。如果相等$(n+m-1)/2$，则该数是中位数。$O((n+m)^2)$

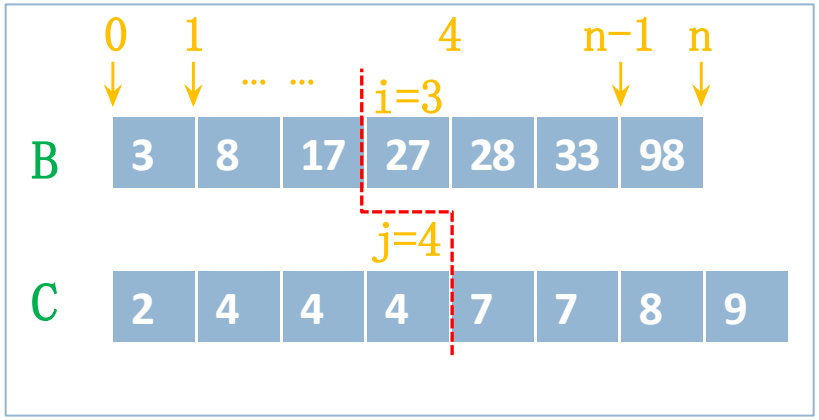
（二）归并法：利用二分归并算法中的合并算法把B、C数组为一个有序数组A，则中位数为A[(n+m-1)/2]。$O(n+m)$

习题课

（三）改进归并法：利用二分归并算法中的合并算法把B、C数组，但只移到指针，数据不移动，当k指针计数到(n+m-1)/2时算法结束，此时对应的i或j即为中位数。$O(n+m)$（空间复杂度$O(1)$）

（四）改进蛮力法：取B的中位i，判断B[i]是不是合并后的中位数，若是算法结束、否则取B[0-(i-1)]或B[(i+1)-(n-1)]的中位数，继续以上判断，直至找到中位数或者下标越界算法结束。$O(\min(n+m))$（空间复杂度$O(1)$）

k=(n+m)/2， j=k-i



满足：i + j == (n+m)/2

B[i] < C[j+1]

C[j] < B[i+1]

中位数：return max(B[i],C[j])

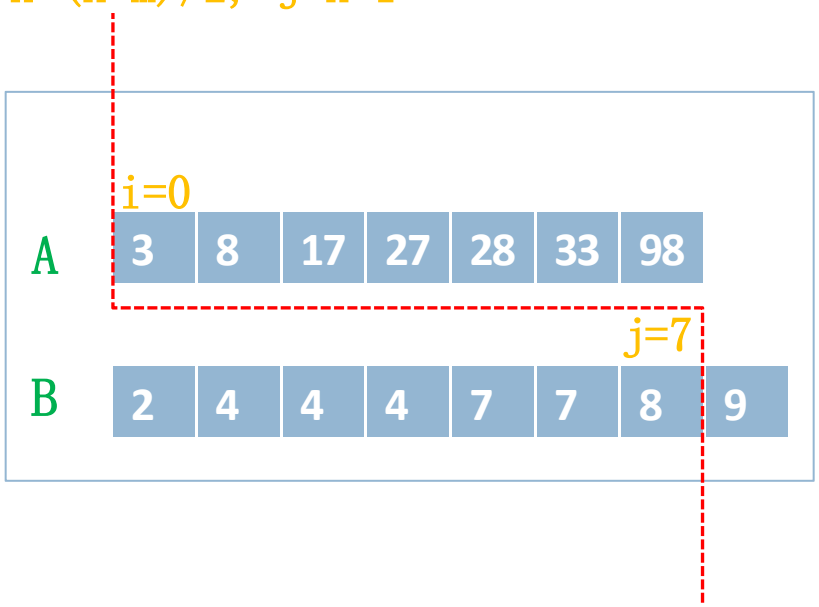
有4种特殊情况：

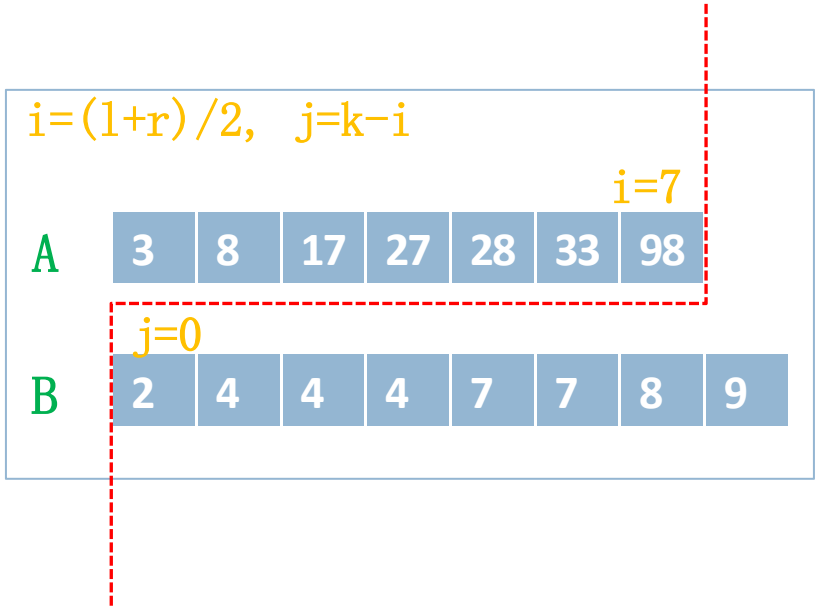
①i==0，不包含A中元素。中位数为B[(n+m-1)/2]

②i==n 且 n==m，全包含A中元素。中位数为min(A[n-1]，B[0])

③i==n，全包含A中元素。中位数为max(min(A[n-1],B[j]))，B[j-1])

k=(n+m)/2，j=k-i





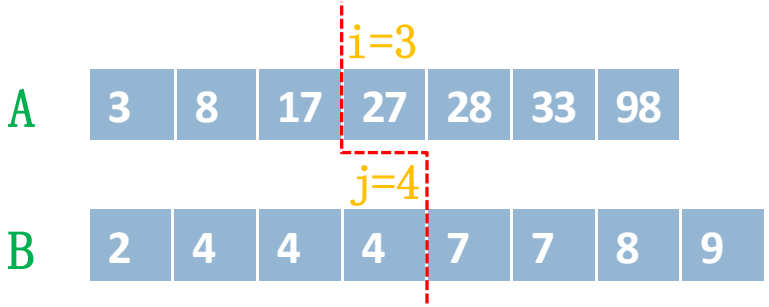
习题课

（五）分治法：取A、B元素数小的作为A，取A的中间位置i划分A的中位线（i范围$left$~$right$+1），则B对应的中位线是$k-i$，判断此划分合不合理？若合理算法结束、否则取$A[left\sim(i-1)]$或$B[(i+1)\sim right)]$的中位线，递归找到A的中位线或者下标越界算法结束。$O(\log(\min(n,m)))$（空间复杂度$O(1)$）

$l \leftarrow 0$，$r=\overset{7}{n}$，$k \leftarrow (n+\overset{7}{m})/2$

折半查找

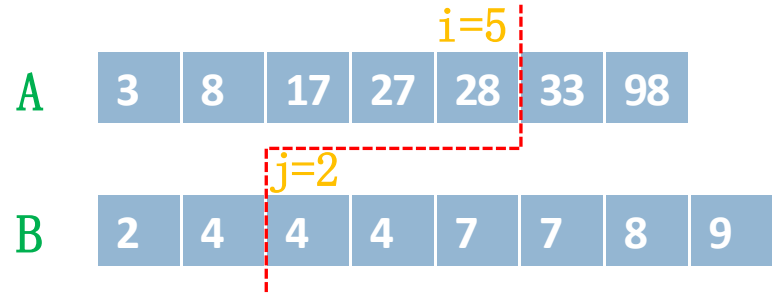
$i=(l+r+1)/2$，$j=k-i$




```
if(i==0)return B[j-1]; //左半部不含B
if i==n && j==0 return A[n-1] //此时m==n或m==n+1
if i==n return max(A[n-1],B[j-1])
if A[i-1]>B[j] then r=i-1
else if B[j-1]>A[i] then l=i+1
else return max(A[i-1],B[j-1])
```

$l=4$，$r=7$，$k=8$

$i=(l+r)/2$，$j=k-i$



C++实现代码：

```
int median(int A[], int B[], int n, int m){
    if( n>m ) return median(B, A, m, n); //选个数小的数组划分
    int l=0, r=n, k=(m+n)/2; //A数组界限、总数组中中位数位置
    int i, j; //A、B数组中位数的位置(A、B数组下标)
    while(true){
        i=(l+r+1)/2, j=k-i;
        if( i==0 ) return B[j-1]; //左半部不含A
        if( i==n && j==0 ) return A[n-1]; //A全在左半部且B数组全在右半部
        if( i==n )return max(A[n-1],B[j-1]);//选A[n-1], B[j-1]中大的

        if( A[i-1]>B[j] ) r=i-1;
        else if( B[j-1]>A[i] ) l=i+1;
        else return max(A[i-1],B[j-1]);
    }
}
```

C++实现代码：

```
#include <iostream>
using namespace std;

int main(){
    int A[] = {2, 4, 4, 4, 7, 7, 8, 9};
    int B[] = {3, 8, 17, 27, 28, 33, 98};
    int n, m, x;
    n = sizeof(A)/sizeof(int);
    m = sizeof(B)/sizeof(int);
    x = median(A, B, n, m);
    cout << "x = " << x << endl << endl;
    return 0;
}
```

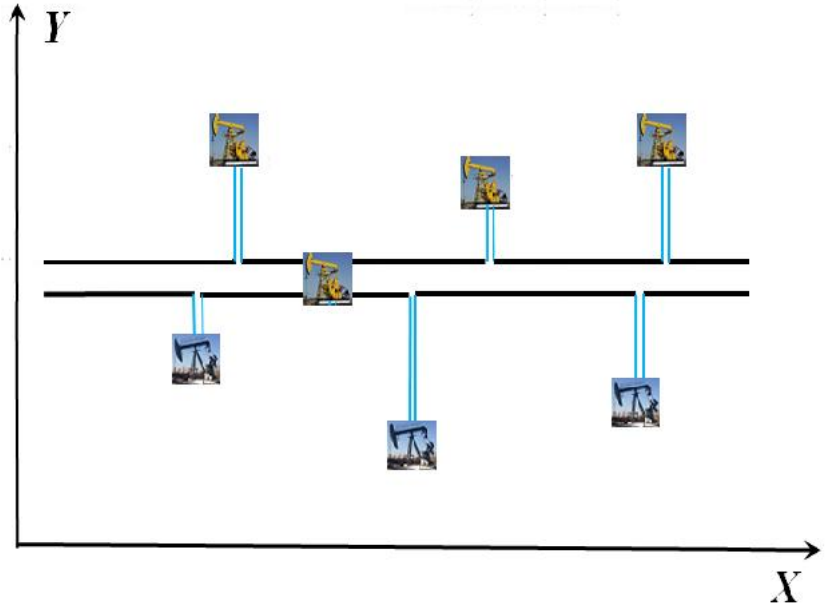
习题课

P50 2.26 某石油公司计划建一条由东向西(x方向)的输油管道，该管道要穿过一个有n口油井的油田，从每口油井都要有一条输油管道，沿最短路径与主管道相连(南北方向)，如果给定n口油井的位置，即它们的x坐标和y坐标，设定一个算法来确定主管道的位置，使得每口油井到主管道之间的输油管道长度之和达到最小。

1.写出解决问题的算法。

2.分析算法的时间复杂度。

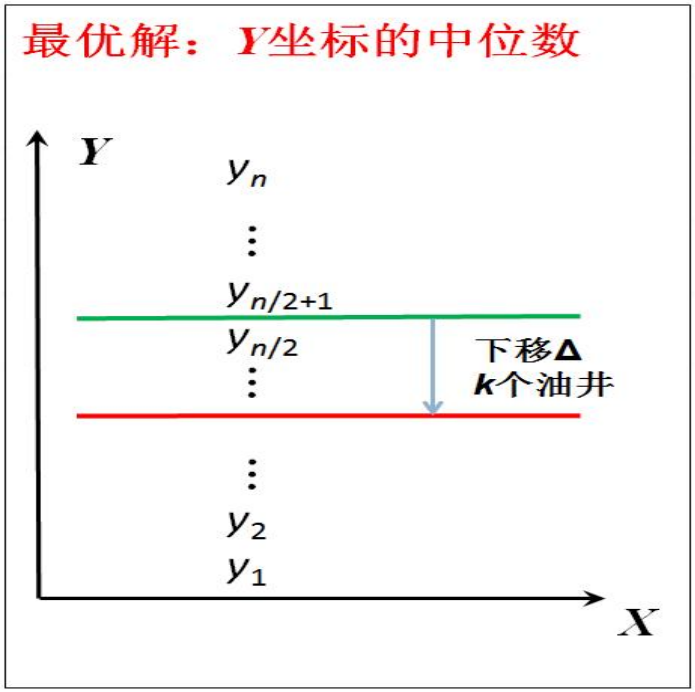
习题课



求证中位数是输油管道的最佳位置

假设输油管道在中位数位置，则上边的油井与下边的油井数量相同。设上边的油井到管道的垂直距离之和为s_1，下边之和为s_2，则总距离为$s_0=s_1+s_2$。

习题课



若管道下移距离Δ越过中位线1个油井Δ'

$$s = (s_1+\Delta*n/2)+(s_2-\Delta*n/2)+\Delta' = s_1+s_2+\Delta' > s_0$$

若管道下移距离Δ越过中位线k个油井Δ_1'、Δ_2' … Δ_k'

$$s = (s_1+\Delta*n/2)+(s_2-\Delta*(n/2-k)+\Delta_1' +\Delta_2' +\ldots+ \Delta_k'$$

$$= s_1+s_2+ \Delta*k+\Delta_1' +\Delta_2' +\ldots+ \Delta_k' > s_0$$

所以，输油管道问题归结为求第小的问题

Select(S,k)，k为中位数

算法复杂的$O(n)$

本节课小结

注意事项：

①正确理解题意

②比较各种算法优劣

③缜密边界条件，写出算法伪代码

③分析算法复杂度

④写出程序代码

结 束

谢谢收看，再见！